

משחקים

משחקים הם דרך טובה להראות יכולות בסביבות מוגדרות ומוכרות.
משחקים יכולים להיות:

- עם מידע מלא או עם מידע חסר
- כאשר יש מידע חסר, רכישת מידע הופכת להיות חלק מהאסטרטגיה.
- דטרמיניסטיים או סטוכאסטיים
- כאשר התוצאות סטוכאסטיות, אין אסטרטגיה רגילה - כי אפשר להחליט על הפעולה הבאה רק כאשר רואים את תוצאות הפעולה הקודמת.
- היריב עצמו הוא לא מכונה הסתברותית - יש לו רצונות והוא דוחף לנצחון.
- הנחת הרציונליות - אנחנו מניחים שהשחקן רוצה לנצח. כלומר - כל פעם הוא מנסה למקסם את הערך שלו.
- נעסוק כאן במשחקים דטרמיניסטיים עם מידע מלא. אם ניתן למדל את עץ המשחק - אין סיבה לשחק, כי שני השחקנים יכולים לעשות את החישוב.
- משפט: במשחק סופי, דטרמיניסטי, ועם מידע מלא - או שלשחקן המתחיל יש אסטרטגיה מנצחת, או שלשחקן השני יש אסטרטגיה מנצחת, או שלשניהם יש אסטרטגיה המובילה לתיקו.
- אנקדוטות: • דמקה הוא משחק תיקו. המשחק פתור - חישבו כבר את כל האפשרויות שלו.
- חושדים ששחמט וגו הם גם משחקי תיקו - אבל הם יותר מדי גדולים מכדי לחשב עד הסוף, אז לא יודעים בוודאות.

אלגוריתם minmax

במשחק סכום אפס, כל פרס¹ ממודל כמספר יחיד. נקרא לשחקן שרוצה למספר את המספר הזה "שחקן max" ולשחקן השני "שחקן min". אם ניתן לבנות את כל העץ, אפשר בכל רמה לבחור את המקסימום או את המינימום (תלוי מי התור). בשביל זה עושים DFS על העץ - כי חייבים להגיע עד הסוף כדי לדעת מה הפרס. את ה DFS עושים עם backward propagation - כלומר מגיעים עד הסוף ומפמפמים למעלה את הערכים שיש בתחתית העץ. סיבוכיות הזמן היא $O(b^m)$ וסיבוכיות המקום היא $O(bm)$ (כאשר b הוא branching factor ו m עומק העץ - כלומר מספר התורות) במקרה של שחמט, $m \approx 100$; $b \approx 35$ - זה יותר מדי², ולכן אי אפשר לפתור עם minmax נאיבי.

¹פרס גם יכול להיות שלילי - כלומר עונש. עדיין נקרא לזה "פרס".
²אבל שחמט הוא עדיין משחק סופי בגלל שיש חוקים שגורמים לו להיות סופי.

$\alpha - \beta$ pruning

שיפור אחד הוא $\alpha - \beta$ pruning. הרעיון הוא לשמור, בזמן פיתוח של כל קוקוד, את ערכי ה- α וה- β שלו - הטווח של הפרסים שאפשר להגיע אליהם. ברגע ששחקן רואה שהערך הכי טוב (מבחינתו) של הקודקוד פחות טוב מקודקוד אחר שהוא יכול לכוון אליו - אפשר לקטום את כל הקודקוד הזה.

α הוא הbest value של max, ו- β הוא הbest value של min. אם הפרסים מסודרים בדרך הטובה ביותר, מכל קודקוד נצטרך לפתח 2 branch. כמות הקודקודים תפחת ל- $O(b^{\frac{m}{2}})$. זה אומר שאנחנו יכולים להסתכל פי 2 יותר עמוק. במקרה הגרוע - זה לא יחסוך לנו כלום.

יוריסטיקות

אם יש לנו פונקציה שמשערכת כמה מצב מסויים רצוי לנו, נוכל להשתמש בה. במקום לרדת עד סוף העץ, נרד מספר קטן של צעדים - ונפעיל את היוריסטיקה על הקודקודים שם בשביל לבחור מה הכי טוב. בשחמט, למשל, אפשר לעשות ווקטור יוריסטיקות:

$$h(s) = w_1 f_1(s) + \dots + w_n f_n(s)$$

כאשר $f_i(s)$ הם פיצ'רים שמסתכלים על מספר הכלים על הלוח - למשל

$$w_1 = 9 \quad f_1(s) = \#(\text{white queens}) - \#(\text{black queens})$$

נשים לב שהיוריסטיקה צריכה להיות מונוטונית - אבל הערכים לא כזה מעניינים אותנו. מה שחשוב זה רק הסדר - מה יותר טוב ממה. במשחקים סטוכסטיים זה כן חשוב, כי עושים ממוצעים, ואז הלינאריות חשובה.

משחקים סטוכאסטיים

המimimax במשחקים סטוכאסטיים נקרא expected minmax כי מסתכלים על התוחלת של כל קודקוד.

במקרה של משחקים לא-דטרמיניסטיים, ההתנהגות של minmax תלויה בלינאריות של היוריסטיקה. אנחנו רוצים, עד כמה שאפשר, שהיוריסטיקה תהיה טרנספורמציה לינארית חיובית של התוחלות האמיתיות - כדי שההתנהגות של minmax תעבוד כמו שצריך כשעושים ממוצעים משוקללים של הפרסים לחשב את התוחלת.

משחקים עם מידע חסר

מידע חסר משפיע על ההסתברויות. בפקר, למשל, זה לא רעיון טוב לחשב את ההסתברות שליריב יש קלפים מסויימים לפי ההסתברות לשלוף אותם מהחפיסה, כי העובדה שהוא ממשיך לשחק נותנת לנו מידע.

אנקדוטה: פוקר בשני שחקנים הוא משחק פתור. בגלל זה, אין סיבה לעשות call כשיש רק שני שחקנים - רק raise או fold.

אנחנו לא דווקא רוצים למדל את היריב בתור הסתברות. היריב יכול לעשות בחירות שיביאו אותנו למקום לא טוב. אם היריב הוא אדוורסריאלי, והוא יודע מה אנחנו עושים, הוא יכול לדפוק אותנו.

Monte Carlo Tree Search

אחד מאלגוריתמי הRLn היותר טובים במשחקים. זהו אלגוריתם שדווקא כן ממדל את השחקן השני בתור הסתברות.

- הרעיון: • נסתכל על כל מהלך בתור יד בmulti armed bandit
- במקום להסתכל על MAB אחד, נבנה עץ סטוכסטי של MAB.
- נשתמש באלגוריתם שעושה איזון בין exploration לexploitation - אלגוריתם UCB1.
- ממדלים גם את עצמנו וגם את היריב בתור הסתברויות.
- ממזערים את regret:

$$\rho = T\mu^* - \sum_{t=1}^T r_t$$

- התגמולים יבואו מסוף העץ.

האלגוריתם עובד בארבעה שלבים:

1. בחירה - בוחרים מצב שייצג את bandit

2. הרחבה - מרחיבים את המצב הנבחר.

3. סימולציה - משחקים משחק שבו בכל שלב בוחרים יוניפורמית.

4. backpropagation - מעדכנים את ערכי המצבים לפי הסימולציה (כה ניצחנו לעומת כמה פעמים שיחקנו)

אפשר להפעיל UCB1 רק אחרי שדגמנו כל יד לפחות פעם אחד, ולכן נתחיל מround robin (נרחיב כל אחת מהידיים)
חוזרים על התהליך ומרחיבים את העץ עד שמישהו מבחין אומר לעצור (למשל - נגמר זמן החישוב) ואז מסתכלים מבחין על העץ ובוחרים את הקודקוד עם הערך הגדול ביותר.
UCB1 שמופעל על עצים נקרא UCT.

בתור RL

אפשר להשתמש בזה לMDP³ כאשר אנחנו לא יודעים את ההסתברויות שלו. זה הופך את זה לRL.

³או למשחק שמודלים כMDP.